DATABASE NORMALIZATION REPORT

DRMS · Disaster Relief Management System

PROJECT	DRMS (Disaster Relief Management System)
SCOPE	Full schema review + 5 migration files
SOURCE SCHEMA	DRMS_SCHEMA.sql
NORMALIZATION VERDICT	3NF — Yes
SYSTEM READINESS	High
MIGRATION IMPACT	Positive, integrity-improving

Prepared as an engineering review of schema design and migration history, evaluating conformance to First, Second, and Third Normal Form.

Contents

Executive Summary

The database schema is best described as being in **Third Normal Form (3NF)** for practical system use.

It satisfies the standard rules of 3NF:

- each table has a primary key;
- all non-key attributes depend on the whole key;
- no non-key attribute depends on another non-key attribute.

The schema is therefore structurally normalized and suitable for a medium-scale operational system.

1. Normalization Assessment

1.1 First Normal Form (1NF)

The schema is in 1NF because:

- each table has a single primary key;
- each column stores atomic values;
- repeated groups are not present as multi-valued columns;
- join/relationship tables are used where many-to-many data exists.

Examples:

- `team_members` stores one membership row per `(team_id, user_id)` pair;
- `disaster_locations` stores one location mapping per `(disaster_id, location_id)` pair;
- `request_items` stores one item request row per `(request_id, item_id)` pair.

1.2 Second Normal Form (2NF)

The schema is in 2NF because:

- the tables are already in 1NF;
- every non-key attribute depends on the entire primary key;
- there are no partial dependencies in the main tables.

Examples:

- `inventory` uses the composite key `(warehouse_id, item_id)` and `quantity` depends on the whole pair;
- `request_items` uses the composite key `(request_id, item_id)` and its attributes depend on the full key;
- `team_members` uses the composite key `(team_id, user_id)` with `member_role` depending on the complete pair.

1.3 Third Normal Form (3NF)

The schema is in 3NF because:

- it is in 2NF;
- no non-key attribute depends on another non-key attribute.

Examples:

- `users.user_id` determines `email`, `full_name`, `password_hash`, `role`, and `phone`;
- `shelters.shelter_id` determines its attributes, not any other non-key descriptor;
- `items.item_id` determines `name`, `category`, and `unit` without transitive dependency.

1.4 Why It Is Not a Perfect Theoretical 3NF Model

Although it is 3NF, the schema still contains a few practical normalization trade-offs.

1   Lookup values are stored as text instead of normalized lookup tables.

- `users.role`
- `disasters.status`
- `victims.status`
- `teams.status`
- `relief_requests.status`
- `distributions.status`

These are not violations of 3NF, but they are less flexible than separate reference tables such as roles, statuses, or categories.

2   locations is flattened.

- `division`, `district`, `upazila`, and `union_name` are all stored in one table.
- This is acceptable for a small application but not as hierarchical as a fully normalized geography model.

3   Repeated textual values exist.

- `category`, `status`, and `team_type` are repeated strings.
- This is common in real applications and does not by itself break 3NF.

Conclusion: the schema is functionally in 3NF and is suitable for a normalized application design.

2. Table-by-Table Normalization Review

`users`

Primary key: `user_id`

Attributes: `full_name`, `email`, `password_hash`, `role`, `phone`

— user_id uniquely identifies a user.

— All attributes depend directly on user_id.

— No non-key attribute depends on another non-key attribute.

STATUS · 3NF COMPLIANT

`locations`

Primary key: `location_id`

Attributes: `division`, `district`, `upazila`, `union_name`

— location_id uniquely identifies each row.

— Each location attribute is tied to the location record itself.

— This table is acceptable in 3NF as a simple location record.

STATUS · 3NF COMPLIANT

`disasters`

Primary key: `disaster_id`

Attributes: `title`, `status`, `start_date`, `created_by_admin_id`

— All fields depend on disaster_id.

— created_by_admin_id is a foreign key to users, which is correct.

STATUS · 3NF COMPLIANT

`disaster_locations`

Composite key: `(disaster_id, location_id)`

— This is a many-to-many relationship table.

— It is fully normalized and standard for relational design.

STATUS · 3NF COMPLIANT

`shelters`

Primary key: `shelter_id`

Attributes: `name`, `address`, `capacity`, `admin_id`, `location_id`

— All attributes depend on shelter_id.

— admin_id and location_id are foreign keys.

STATUS · 3NF COMPLIANT

warehouses

Primary key: **warehouse_id**

Attributes: **name**, **admin_id**, **location_id**

— All attributes depend on warehouse_id.

STATUS · 3NF COMPLIANT

items

Primary key: **item_id**

Attributes: **name**, **category**, **unit**

— name, category, and unit all depend on item_id.

STATUS · 3NF COMPLIANT

inventory

Primary key: **inventory_id**

Composite key: **(warehouse_id, item_id)**

Attributes: **warehouse_id**, **item_id**, **quantity**

— quantity depends on the pair (warehouse_id, item_id).

— inventory_id is a surrogate key but the real logical key is the pair.

— This is still normalized because no transitive dependency exists.

STATUS · 3NF COMPLIANT

victims

Primary key: **victim_id**

Attributes: **full_name**, **date_of_birth**, **gender**, **priority_level**, **status**, **shelter_id**

— All attributes depend on victim_id.

— shelter_id references a shelter.

STATUS · 3NF COMPLIANT

donations

Primary key: **donation_id**

Attributes: **donor_id**, **warehouse_id**, **item_id**, **quantity**, **donated_at**

— All data depends on donation_id.

— This is a proper transaction table.

STATUS · 3NF COMPLIANT

teams

Primary key: `team_id`

Attributes: `team_name`, `team_type`, `status`, `leader_id`, `approved_by_admin_id`

— All attributes depend on team_id.

— This is consistent with 3NF.

STATUS · 3NF COMPLIANT

team_members

Primary key: `team_member_id`

Composite key: `(team_id, user_id)`

Attributes: `team_id`, `user_id`, `member_role`

— member_role depends on the full membership pair.

— This is a normalized association table.

STATUS · 3NF COMPLIANT

relief_requests

Primary key: `request_id`

Attributes: `shelter_id`, `requested_by_admin_id`, `status`, `requested_at`

— All non-key fields depend on request_id.

STATUS · 3NF COMPLIANT

request_items

Primary key: `request_item_id`

Attributes: `request_id`, `item_id`, `quantity_requested`, `quantity_dispatched`

— Each attribute depends on the request/item row.

— Proper normalization for order detail lines.

STATUS · 3NF COMPLIANT

distributions

Primary key: `distribution_id`

Attributes: `request_id`, `warehouse_id`, `assigned_team_id`, `assigned_by_admin_id`, `status`, `distributed_at`

— All values depend on the distribution record itself.

— This is a standard operational table.

STATUS · 3NF COMPLIANT

3. Migration Impact on Normalization

The migration files show a database that evolved during development, and the migrations generally support normalization by enforcing integrity and consistency.

Migration 001 — single_team_membership.sql

Purpose — Ensure each user belongs to at most one team.

Normalization effect:

- prevents redundant membership duplication;
- preserves the logical uniqueness of team membership;
- improves data integrity and avoids update anomalies.

This supports a cleaner relational design.

Migration 002 — team_review_remark.sql

Purpose — Adds a review remark to teams.

Normalization effect:

- this is a local attribute addition to the team entity;
- it does not violate 3NF because the remark is directly tied to `team_id`.

Migration 003 — victim_disaster_relationship.sql

Purpose — Links each victim to the disaster associated with their relief case.

Normalization effect:

- improves relational correctness by representing the relationship explicitly;
- avoids storing disaster information redundantly inside victims;
- supports a clean association between victims and disasters.

This is a normalization-positive change.

Migration 004 — victim_status_values.sql

Purpose — Standardizes victim statuses to allowed values.

Normalization effect:

- reduces inconsistent free-text values;
- improves reliability of lookup semantics;
- prevents anomalies caused by invalid or divergent status labels.

This is a data-quality improvement that strengthens the normalized design.

Migration 005 — ensure_users_full_name.sql

Purpose — Ensures the `full_name` field exists and the legacy `name` field is migrated.

Normalization effect:

- resolves schema inconsistency;
- standardizes attribute naming;
- prevents duplicate or ambiguous user identity fields.

This helps preserve a consistent attribute set and reduces semantic anomalies.

4. Overall Conclusion

The schema is aligned with Third Normal Form and is a robust normalized design for a disaster-relief management system.

Strengths

- clear entity boundaries;
- use of primary and foreign keys;
- association tables for many-to-many relationships;
- logical decomposition of operational data.

Limitations

Mostly practical, and not 3NF violations:

- some values are text-coded rather than stored in lookup tables;
- the geography model is simplified;
- certain domain values repeat across tables.

Overall Verdict

3NF	Yes
Practical system readiness	High
Migration impact	Positive and integrity-improving

Final Statement

This schema is a solid 3NF-compliant relational database design with good integrity controls and migration support. It is suitable for continued development with only modest refinement for domain lookup tables and location hierarchy modeling.